



Compte Rendu des TPs

Cycle d'Ingénieurs
Ingénierie des Systèmes d'Information et Big Data
S6

présenté par
Asmaa ZAGHIGH

À
L'École Nationale des Sciences Appliquées de Berrechid
Université Hassan 1^{er}
-SETTAT-

Année Universitaire : 2025-2026

TP des Sous Requete

```
1 EXERCICE 1:
2 -- 1.
3 SELECT Numetu, Nometu, Cdsexe
4 FROM ETUDIANT
5 WHERE Dtnaiss IS NULL;
6
7 -- 2.
8 SELECT Nometu, Dtnaiss
9 FROM ETUDIANT
10 WHERE Nometu LIKE '_a%';
11
12 -- 3.
13 SELECT Nomat
14 FROM MATIERE
15 WHERE Coeff BETWEEN 1 AND 2;
16
17 -- 4.
18 SELECT N1.Note
19 FROM NOTES N1
20 WHERE N1.Numetu = 1
21     AND N1.Note IN (
22         SELECT N2.Note
23         FROM NOTES N2
24         WHERE N2.Numetu = 2
25     );
26
27 -- 5.
28 SELECT N1.Note
29 FROM NOTES N1
30 WHERE N1.Numetu = 1
31     AND N1.Note > ANY (
32         SELECT N2.Note
33         FROM NOTES N2
34         WHERE N2.Numetu = 2
35     );
36
37 -- 6.
38 SELECT N1.Note
39 FROM NOTES N1
40 WHERE N1.Numetu = 1
41     AND N1.Note < ALL (
42         SELECT N9.Note
43         FROM NOTES N9
44         WHERE N9.Numetu = 9
45     );
46
47 -- 7.
48 SELECT E.*
49 FROM ETUDIANT E
50 WHERE E.Numetu NOT IN (
51     SELECT DISTINCT Numetu
52     FROM NOTES
```

```

53 );
54
55 -- 8.
56 SELECT S.Lbsexe,
57         AVG(DATEDIFF('2000-01-01', E.Dtnaiss) / 365) AS
           AgeMoyen
58 FROM ETUDIANT E
59 JOIN SEXE S ON E.Cdsexe = S.Cdsexe
60 GROUP BY S.Lbsexe;
61
62 -- 9.
63 SELECT EN.Nomens, EN.Grade
64 FROM ENSEIGNANT EN
65 JOIN MATIERE M ON EN.Numens = M.Numens
66 WHERE UPPER(M.Nomat) = 'HISTOIRE';
67
68 -- 10.
69 SELECT E.Numetu, E.Nometu
70 FROM ETUDIANT E
71 WHERE E.Numetu NOT IN (
72     SELECT N.Numetu
73     FROM NOTES N
74     JOIN MATIERE M ON N.Numat = M.Numat
75     WHERE UPPER(M.Nomat) = 'SOCIOLOGIE'
76 );
77
78
79 EXERCICE 2
80
81 -- 11.
82 SELECT Lieu
83 FROM DEPT
84 WHERE NumDept IN (
85     SELECT NumDept
86     FROM EMP
87     WHERE Commission IS NOT NULL
88     AND Commission > 0
89 );
90
91 -- 12.
92 SELECT DISTINCT D.NomDept, D.Lieu
93 FROM DEPT D
94 JOIN EMP E ON D.NumDept = E.NumDept
95 WHERE E.Poste = 'INGENIEUR';
96
97 -- 13.
98 SELECT D.NomDept, D.Lieu
99 FROM DEPT D
100 WHERE D.NumDept NOT IN (
101     SELECT NumDept
102     FROM EMP
103     WHERE Poste = 'INGENIEUR'
104 );

```

```

105
106 -- 14.
107 SELECT E.Matr, E.NomE, E.Salaire
108 FROM EMP E
109 WHERE E.Salaire < 0.5 * (
110     SELECT SUP.Salaire
111     FROM EMP SUP
112     WHERE SUP.Matr = E.Sup
113 );
114
115 -- 15.
116 SELECT NomE
117 FROM EMP
118 WHERE Salaire > ALL (
119     SELECT Salaire
120     FROM EMP
121     WHERE Poste = 'COMMERCIAL'
122 );
123
124 -- 16a.
125 SELECT NomE
126 FROM EMP E
127 WHERE E.Salaire > ALL (
128     SELECT Salaire
129     FROM EMP
130     WHERE Poste = 'COMMERCIAL'
131     AND NumDept = E.NumDept
132 );
133
134 -- 16b.
135 SELECT NomE
136 FROM EMP E
137 WHERE E.NumDept IN (
138     SELECT NumDept FROM EMP WHERE Poste = 'COMMERCIAL'
139 )
140 AND E.Salaire > ALL (
141     SELECT Salaire
142     FROM EMP
143     WHERE Poste = 'COMMERCIAL'
144     AND NumDept = E.NumDept
145 );
146
147 -- 17.
148 SELECT NomE
149 FROM EMP
150 WHERE (Poste, Sup) IN (
151     SELECT Poste, Sup
152     FROM EMP
153     WHERE NomE = 'BERGER'
154 )
155 AND NomE <> 'BERGER';

```

TP Vues

```
1  -- PARTIE 1          TRAVAIL DEMAND SUR LES VUES
2
3  -- 1.
4
5  CREATE VIEW empxpublic AS
6  SELECT Matr, NomE, Poste, DatEmb, Sup, Commission, NumDept
7  FROM EMP;
8
9  -- V rification
10 SELECT * FROM empxpublic;
11
12
13 -- 2.
14 Non, une vue classique n'am liore PAS les performances car
    Une vue classique (ou "vue logique") n'est qu'une
    requ te stock e et a chaque appel, le SGBD r ex cute
    la requ te sous-jacente comme si elle avait t
    crite directement. Il n'y a aucune mise en cache
15 du r sultat.
16
17 - En revanche, une vue peut INDIRECTEMENT aider les
    performances si :
18     * Elle permet au SGBD d'appliquer des optimisations (
        query rewriting)
19     * Elle r duit la complexit des requ tes crites par
        les d veloppeurs
20     (moins d'erreurs, requ tes mieux structur es)
21     * Elle limite les colonnes renvoy es (SELECT r duit le
        volume transf r )
22
23 Conclusion : vue classique = s curit + lisibilit , PAS
    de gain de performance.
24
25
26 -- EXERCICE 6
27
28 -- Q1. Architecture de vues pour r pondre aux contraintes de
    s curit
29
30
31 Vue pour les MANAGERS : que les employ s de leur
    d partement
32 Le manager est identifi par son NumDept, pass en
    param tre ou filtr
33 On cr e une vue par d partement .
34 -- Exemple vue pour le d partement 10) :
35 CREATE VIEW vue_manager_dept10 AS
36 SELECT Matr, NomE, Poste, DatEmb, Sup, NumDept
37 FROM EMP
38 WHERE NumDept = 10;
39
40 -- Vue pour tout manager (sans le salaire des autres) :
```

```

41 CREATE VIEW vue_manager AS
42 SELECT Matr, NomE, Poste, DatEmb, Sup, NumDept
43 FROM EMP;
44 -- => On accordera l'acc s cette vue uniquement aux
    managers,
45 -- en leur refusant l'acc s la table EMP directement (
    REVOKE).
46
47 -- Vue pour le service RH : acc s complet toutes les
    informations
48 CREATE VIEW vue_rh AS
49 SELECT E.Matr, E.NomE, E.Poste, E.DatEmb, E.Sup,
50        E.Salaire, E.Commission, E.NumDept,
51        D.NomDept, D.Lieu
52 FROM EMP E
53 JOIN DEPT D ON E.NumDept = D.NumDept;
54
55 -- Vue pour les EMPLOY S : chaque employ voit ses propres
    infos seulement
56 --
57 -- Exemple sous Oracle / PostgreSQL :
58 CREATE VIEW vue_employe AS
59 SELECT Matr, NomE, Poste, DatEmb, NumDept, Salaire
60 FROM EMP
61 WHERE NomE = CURRENT_USER;
62
63 -- Attribution des droits selon les r les :
64 GRANT SELECT ON vue_manager TO role_manager;
65 GRANT SELECT ON vue_rh TO role_rh;
66 GRANT SELECT ON vue_employe TO role_employe;
67 REVOKE SELECT ON EMP FROM role_manager, role_employe;
68
69
70 -- Q2.
71 /*
72  LIMITES DES VUES CLASSIQUES :
73
74  1. R valuation syst matique
75     - chaque appel, toute la requ te est r ex cut e
76       sur la table source.
77     - Sur des centaines de milliers de lignes, cela peut
78       tre tr s lent.
79
80  2. Pas de stockage physique du r sultat
81     - Aucun r sultat n'est mis en cache entre deux appels.
82     - Chaque requ te sur la vue repart de z ro.
83
84  3. Impossible d'indexer une vue classique directement
85     - On ne peut pas cr er un index sur une vue simple.
86     - Les index de la table sous-jacente s'appliquent, mais
87       pas toujours
88       de mani re optimale selon la r criture de la
89       requ te.

```

4. Jointures complexes mal optimisées
 - Une vue qui joint plusieurs grandes tables peut produire des plans d'exécution très coûteux (full table scans).
5. Vues imbriquées
 - Une vue qui appelle une autre vue (imbrication) empire les performances car les optimiseurs ont du mal à "aplatir" la requête.

Conclusion : pour un grand volume de données, la vue classique reste un outil de sûreté et de simplicité, pas de performance.

*/

-- Q3

/*

UNE VUE MATERIALISÉE (MATERIALIZED VIEW) stocke physiquement le résultat de la requête dans une table temporaire persistante.

AVANTAGES dans un contexte de grand volume :

1. Performances en lecture drastiquement améliorées
 - Le résultat est précalculé. Une requête sur la vue matérialisée ne réexécute pas les jointures ou agrégations coûteuses.
2. Indexable
 - On peut créer des index sur une vue matérialisée exactement comme sur une table ordinaire.
3. Idéale pour les rapports et statistiques RH
 - Ex. : nombre d'employés par département, masse salariale par site, etc. Résultats précalculés, consultables instantanément.

INCONVÉNIENTS :

1. Données potentiellement périmées (stale data)
 - La vue matérialisée doit être rafraîchie (REFRESH) pour refléter les mises à jour de la table source.
 - Selon la stratégie : REFRESH ON COMMIT (immédiat,

```

126         co teux)
        ou REFRESH ON DEMAND / SCHEDULED (p riodique , moins
127         co teux).
128
129 2. Espace disque suppl mentaire
130     - Les donn es sont dupliqu es physiquement.
131
132 3. Complexit de maintenance
133     - Il faut g rer la politique de rafra chissement.
134
135 */
136
137 -- =====
138 -- Q4. Vues vs m canismes GRANT / REVOKE
139 -- =====
140 /*
141     COMPARAISON VUES          GRANT / REVOKE
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162

```

Crit re / REVOKE	VUES	GRANT
Granularit , colonne (limit)	Colonne, ligne, jointure	Table
Filtrage lignes pas natif SQL std)	Oui (WHERE dans la vue)	Non (
Masquage donn es Partiel (GRANT col)	Oui (exclure colonnes)	
Complexit Simple, d claratif	N cessite de cr er et maintenir les vues	
Portabilit	Bonne	Bonne
Performance	Neutre (vue classique)	Neutre

```

163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
269
```



```

163  - On R VOQUE l'acc s direct EMP pour tous les
      utilisateurs.
164  - On ACCORDE l'acc s uniquement aux vues adapt es
      chaque r le.
165  */
166
167  REVOKE ALL ON EMP FROM PUBLIC;
168  REVOKE ALL ON DEPT FROM PUBLIC;
169
170  GRANT SELECT ON vue_rh TO role_rh;
171  GRANT SELECT ON vue_manager TO role_manager;
172  GRANT SELECT ON vue_employe TO role_employe;
173
174  -- Q5. Une vue peut-elle garantir elle seule la s curit
      compl te ?
175
176  /*
177  NON, une vue seule ne suffit PAS garantir la s curit
      compl te.
178
179  Raisons :
180
181  1. Si l'acc s direct la table n'est pas r voqu (
      REVOKE),
182  l'utilisateur peut contourner la vue et interroger la
      table
183  directement les vues sans REVOKE ne servent rien
      en s curit .
184
185  2. Injection SQL
186  - Une vue ne prot ge pas contre les injections SQL si l'
      application
187  cliente construit des requ tes dynamiques non
      s curis es.
188
189  Conclusion : la s curit d'un SGBD repose sur une
      combinaison de :
190  Vues (filtrage logique)
191  GRANT / REVOKE (permissions)
192  Authentification forte
193  S paration des r les (principe du moindre privil ge
      )
194  */
195
196
197  -- Q6. Strat gie d'optimisation compl te
198
199  /*
200  STRAT GIE D'OPTIMISATION POUR UNE BASE GRAND VOLUME
201
202  1. INDEX
203  - Cr er des index sur les colonnes fr quemment
      utilis es en WHERE,

```

```

204 JOIN, ORDER BY.
205 - Index composites pour les requêtes multi-colonnes.
206 - viter les index sur des colonnes faible
    cardinalit (ex: Cdsexe).
207 */
208
209 CREATE INDEX idx_emp_numdept ON EMP(NumDept);
210 CREATE INDEX idx_emp_poste ON EMP(Poste);
211 CREATE INDEX idx_emp_sup ON EMP(Sup);
212 CREATE INDEX idx_emp_dept_poste ON EMP(NumDept, Poste); --
    index composite
213
214 /*
215 2. PARTITIONNEMENT
216 - Partitionner la table EMP par NumDept ou par Lieu (DEPT
    )
217 pour que les requêtes d'un manager ne lisent que la
    partition
218 de son département (partition pruning).
219 - Partitionnement par plage de dates (DatEmb) pour les
    archives.
220 */
221
222
223 CREATE TABLE EMP_DEPT10 PARTITION OF EMP_PARTITIONED FOR
    VALUES IN (10);
224 CREATE TABLE EMP_DEPT20 PARTITION OF EMP_PARTITIONED FOR
    VALUES IN (20);
225 CREATE TABLE EMP_DEPT30 PARTITION OF EMP_PARTITIONED FOR
    VALUES IN (30);
226
227 /*
228 3. VUES MATÉRIALISÉES
229 - Pr calculer les agrégats coûteux (masse salariale,
    effectifs,
230 moyennes par département) dans des vues
    matérialisées.
231 - Rafraîchissement planifié (ex: chaque nuit) pour les
    rapports RH.
232 */
233
234 CREATE MATERIALIZED VIEW vue_stats_dept AS
235 SELECT D.NumDept, D.NomDept, D.Lieu,
236        COUNT(E.Matr) AS NbEmployes,
237        AVG(E.Salaire) AS SalaireMoyen,
238        MAX(E.Salaire) AS SalaireMax,
239        MIN(E.Salaire) AS SalaireMin,
240        SUM(E.Salaire) AS MasseSalariale
241 FROM DEPT D
242 LEFT JOIN EMP E ON D.NumDept = E.NumDept
243 GROUP BY D.NumDept, D.NomDept, D.Lieu;
244
245 CREATE INDEX idx_vm_stats_dept ON vue_stats_dept(NumDept);

```

246
247
248
249
250
251
252
253
254
255
256
257
258
259
260

/*

SYNTH SE DE LA STRAT GIE :

Besoin

Solution

S curit par r le

Vues + GRANT/REVOKE

*Performances en lecture
mat rialis es
Requ tes par d partement
index composite*

Index + Vues

*Rapports RH p riodiques
REFRESH planifi*

Partitionnement +

*Tra abilit des acc s
ou outils SGBD)*

Vues mat rialis es +

Audit logs (triggers

*S curit des donn es
authentification forte*

Chiffrement +

*/

TP PL/SQL

```
1  -- EXERCICE 1
2
3
4  CREATE TABLE Nombre (
5      n NUMBER
6  );
7
8  BEGIN
9      FOR i IN 1..100 LOOP
10         INSERT INTO Nombre VALUES(i);
11     END LOOP;
12     COMMIT;
13 END;
14 /
15
16 -- EXERCICE 2
17
18 DECLARE
19     somme NUMBER := 0;
20 BEGIN
21     FOR i IN 1..1000 LOOP
22         somme := somme + i;
23     END LOOP;
24
25     DBMS_OUTPUT.PUT_LINE('Somme = ' || somme);
26 END;
27 /
28
29 -- EXERCICE 3
30
31 BEGIN
32     FOR i IN 1..50 LOOP
33         IF MOD(i,2)=0 THEN
34             DBMS_OUTPUT.PUT_LINE(i);
35         END IF;
36     END LOOP;
37 END;
38 /
39
40 -- EXERCICE 4
41
42 DECLARE
43     n NUMBER := 5;
44     fact NUMBER := 1;
45 BEGIN
46     FOR i IN 1..n LOOP
47         fact := fact * i;
48     END LOOP;
49
50     DBMS_OUTPUT.PUT_LINE('Factoriel de ' || n || ' = ' || fact);
51 END;
52 /
```

```

53
54 -- EXERCICE 5
55
56 CREATE TABLE Etudiant (
57     id NUMBER PRIMARY KEY,
58     nom VARCHAR2(50),
59     note NUMBER(5,2)
60 );
61
62 BEGIN
63     FOR i IN 1..10 LOOP
64         INSERT INTO Etudiant
65             VALUES(
66                 i,
67                 'Etudiant_'||i,
68                 10 + i
69             );
70     END LOOP;
71
72     COMMIT;
73 END;
74 /
75
76 -- EXERCICE 6
77
78 DECLARE
79     i NUMBER := 10;
80 BEGIN
81     WHILE i >= 1 LOOP
82         DBMS_OUTPUT.PUT_LINE(i);
83         i := i - 1;
84     END LOOP;
85 END;
86 /
87
88 -- EXERCICE 7
89
90 CREATE TABLE Produit (
91     id NUMBER PRIMARY KEY,
92     nom VARCHAR2(50),
93     prix NUMBER(10,2)
94 );
95
96 INSERT INTO Produit VALUES (1,'Clavier',100);
97 INSERT INTO Produit VALUES (2,'Souris',40);
98 INSERT INTO Produit VALUES (3,'Ecran',800);
99 COMMIT;
100
101 BEGIN
102     UPDATE Produit
103     SET prix = prix * 1.10;
104
105     COMMIT;

```

```

106         DBMS_OUTPUT.PUT_LINE('Prix augment s de 10 %');
107     END;
108 /
109
110
111 -- EXERCICE 8
112
113 BEGIN
114     DELETE FROM Produit
115     WHERE prix < 50;
116
117     COMMIT;
118
119     DBMS_OUTPUT.PUT_LINE('Produits supprim s');
120 END;
121 /
122 -- EXERCICE 9
123
124 DECLARE
125     n NUMBER := -5;
126 BEGIN
127     IF n > 0 THEN
128         DBMS_OUTPUT.PUT_LINE('Positif');
129     ELSIF n < 0 THEN
130         DBMS_OUTPUT.PUT_LINE('N gatif');
131     ELSE
132         DBMS_OUTPUT.PUT_LINE('Nul');
133     END IF;
134 END;
135 /
136
137 -- EXERCICE 10
138
139 CREATE TABLE Carre (
140     nombre NUMBER,
141     carre NUMBER
142 );
143
144 BEGIN
145     FOR i IN 1..20 LOOP
146         INSERT INTO Carre
147         VALUES(i, i*i);
148     END LOOP;
149
150     COMMIT;
151
152     DBMS_OUTPUT.PUT_LINE('Insertion termin e');
153 END;
154 /

```

TP Record

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     -- 1.
5     TYPE t_employe IS RECORD (
6         matr      NUMBER(5),
7         nom        VARCHAR2(50),
8         poste      VARCHAR2(30),
9         numdept    NUMBER(3),
10        salaire    NUMBER(10, 2)
11    );
12    -- 2.
13    TYPE t_tab_employes IS TABLE OF t_employe
14        INDEX BY PLS_INTEGER;
15
16    -- Declaration de la variable tableau
17    tab_employes    t_tab_employes;
18
19    -- Variables pour les calculs
20    v_total_salaire  NUMBER(12, 2) := 0;
21    v_salaire_moyen  NUMBER(10, 2) := 0;
22    v_salaire_max     NUMBER(10, 2) := 0;
23    v_nb_employes    PLS_INTEGER   := 0;
24
25 BEGIN
26     -- 3. Ins rer
27
28     -- Employ 1
29     tab_employes(1).matr      := 101;
30     tab_employes(1).nom       := 'ASMAA';
31     tab_employes(1).poste     := 'INGENIEUR';
32     tab_employes(1).numdept   := 10;
33     tab_employes(1).salaire   := 3500.00;
34
35     -- Employ 2
36     tab_employes(2).matr      := 102;
37     tab_employes(2).nom       := 'DOUAA';
38     tab_employes(2).poste     := 'ANALYSTE';
39     tab_employes(2).numdept   := 20;
40     tab_employes(2).salaire   := 4200.00;
41
42     -- Employ 3
43     tab_employes(3).matr      := 103;
44     tab_employes(3).nom       := 'SARA';
45     tab_employes(3).poste     := 'COMMERCIAL';
46     tab_employes(3).numdept   := 10;
47     tab_employes(3).salaire   := 2800.00;
48
49     -- Employ 4
50     tab_employes(4).matr      := 104;
51     tab_employes(4).nom       := 'YOUSSEF';
52     tab_employes(4).poste     := 'MANAGER';
```

```

53 tab_employes(4).numdept := 30;
54 tab_employes(4).salaire := 5100.00;
55
56 -- Employ 5
57 tab_employes(5).matr := 105;
58 tab_employes(5).nom := 'YAHYA';
59 tab_employes(5).poste := 'TECHNICIEN';
60 tab_employes(5).numdept := 20;
61 tab_employes(5).salaire := 2200.00;
62
63
64 -- Affichage du contenu du tableau
65 DBMS_OUTPUT.PUT_LINE('
=====');
66 DBMS_OUTPUT.PUT_LINE(' LISTE DES EMPLOY S
ENREGISTR S ');
67 DBMS_OUTPUT.PUT_LINE('
=====');
68 DBMS_OUTPUT.PUT_LINE(
69 RPAD('Matr', 6) || RPAD('Nom', 12) ||
70 RPAD('Poste', 14) || RPAD('Dept', 6) || 'Salaire'
71 );
72 DBMS_OUTPUT.PUT_LINE('
-----');
73
74 v_nb_employes := tab_employes.COUNT;
75
76 FOR i IN 1 .. v_nb_employes LOOP
77
78 -- Affichage de chaque employ
79 DBMS_OUTPUT.PUT_LINE(
80 RPAD(tab_employes(i).matr, 6) ||
81 RPAD(tab_employes(i).nom, 12) ||
82 RPAD(tab_employes(i).poste, 14) ||
83 RPAD(tab_employes(i).numdept, 6) ||
84 tab_employes(i).salaire || ' '
85 );
86 -- 4. Calculer le salaire moyen (cumul total)
87
88 v_total_salaire := v_total_salaire + tab_employes(i).
salaire;
89
90 -- 5. Identifier le salaire maximum
91 IF tab_employes(i).salaire > v_salaire_max THEN
92 v_salaire_max := tab_employes(i).salaire;
93 END IF;
94
95 END LOOP;
96
97 -- Calcul de la moyenne
98 v_salaire_moyen := v_total_salaire / v_nb_employes;
99
100

```



```

101 DBMS_OUTPUT.PUT_LINE('
=====');
102 DBMS_OUTPUT.PUT_LINE(' R SULTATS
');
103 DBMS_OUTPUT.PUT_LINE('
=====');
104 DBMS_OUTPUT.PUT_LINE('Nombre d''employ s : ' ||
v_nb_employes);
105 DBMS_OUTPUT.PUT_LINE('Total des salaires : ' ||
v_total_salaire || ' ');
106 DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' ||
v_salaire_moyen || ' ');
107 DBMS_OUTPUT.PUT_LINE('Salaire maximum : ' ||
v_salaire_max || ' ');
108 DBMS_OUTPUT.PUT_LINE('
=====');
109
110 -- Identifier et afficher quel employ a le salaire
maximum
111
112 FOR i IN 1 .. v_nb_employes LOOP
113 IF tab_employes(i).salaire = v_salaire_max THEN
114 DBMS_OUTPUT.PUT_LINE(
115 '=> Employ le mieux pay : ' ||
116 tab_employes(i).nom || ' (' || tab_employes(i)
.poste || ')' ||
117 ' ' || tab_employes(i).salaire || ' '
118 );
119 END IF;
120 END LOOP;
121
122 DBMS_OUTPUT.PUT_LINE('
=====');
123
124 END;
125 /

```

TP Triggers

```
1  -- CONTRAINTE 10 :
2
3  CREATE OR REPLACE TRIGGER trg_verif_module_valide
4  BEFORE INSERT ON inscription
5  FOR EACH ROW
6  DECLARE
7      v_note_min      module.note_min%TYPE;
8      v_moyenne_notes NUMBER;
9      v_nb_examens    NUMBER;
10     v_nb_passes      NUMBER;
11
12 BEGIN
13     -- R cup rer la note minimale de validation du module
14     SELECT note_min
15     INTO    v_note_min
16     FROM    module
17     WHERE   code_module = :NEW.code_module;
18
19     -- Compter le nombre d'examens du module
20     SELECT COUNT(*)
21     INTO    v_nb_examens
22     FROM    examen
23     WHERE   code_module = :NEW.code_module;
24
25     -- Si aucun examen n'existe pour ce module, on laisse
26     passer
27     IF v_nb_examens = 0 THEN
28         RETURN;
29     END IF;
30
31     -- Compter combien d'examens l' tudiant a pass pour ce
32     module
33     SELECT COUNT(*)
34     INTO    v_nb_passes
35     FROM    passer p
36     JOIN    examen e ON p.id_examen = e.id_examen
37     WHERE   e.code_module = :NEW.code_module
38             AND p.id_etudiant = :NEW.id_etudiant;
39
40     -- Si l' tudiant a pass tous les examens du module,
41     -- calculer sa moyenne pond r e par les coefficients
42     IF v_nb_passes = v_nb_examens THEN
43
44         SELECT SUM(p.note * e.coefficient) / SUM(e.coefficient
45             )
46         INTO    v_moyenne_notes
47         FROM    passer p
48         JOIN    examen e ON p.id_examen = e.id_examen
49         WHERE   e.code_module = :NEW.code_module
50                 AND p.id_etudiant = :NEW.id_etudiant;
51
52         -- Si la moyenne d passe ou gale la note minimale
```

```

=> module valid
50 IF v_moyenne_notes >= v_note_min THEN
51     RAISE_APPLICATION_ERROR(
52         -20010,
53         'ERREUR C10 : L'' tudiant ' || :NEW.
           id_etudiant ||
54         ' a d j valid le module ' || :NEW.
           code_module ||
55         ' avec une moyenne de ' || ROUND(
           v_moyenne_notes, 2) ||
56         ' (note min = ' || v_note_min || ').
           Inscription refus e.'
57     );
58 END IF;
59
60 END IF;
61
62 EXCEPTION
63     WHEN NO_DATA_FOUND THEN
64         -- Le module n'existe pas : la contrainte FK le
           g rera
65         NULL;
66 END trg_verif_module_valide;
67 /
68
69 -- CONTRAINTE 11 :
70 CREATE OR REPLACE TRIGGER trg_verif_conflit_examen
71 BEFORE INSERT ON examen
72 FOR EACH ROW
73 DECLARE
74     v_nb_conflits NUMBER;
75
76 BEGIN
77     -- V rifier s'il existe un examen du m me module
78     -- dont la date est dans l'intervalle [date-7, date+7]
79     SELECT COUNT(*)
80     INTO    v_nb_conflits
81     FROM    examen
82     WHERE   code_module = :NEW.code_module
83           AND ABS(:NEW.date_examen - date_examen) <= 7;
84
85     IF v_nb_conflits > 0 THEN
86         RAISE_APPLICATION_ERROR(
87             -20011,
88             'ERREUR C11 : Un examen pour le module ' || :NEW.
               code_module ||
89             ' existe d j dans une fen tre de 7 jours
               autour du ' ||
90             TO_CHAR(:NEW.date_examen, 'DD/MM/YYYY') ||
91             '. Veuillez choisir une autre date.'
92         );
93     END IF;
94

```

```

95 END trg_verif_conflit_examen;
96 /
97
98 -- CONTRAINTE 12 :
99 CREATE OR REPLACE TRIGGER trg_calcul_moyenne_etudiant
100 AFTER INSERT OR UPDATE ON passer
101 FOR EACH ROW
102 DECLARE
103     -- Nombre total d'examens de tous les modules o l'
104     tudiant est inscrit
105     v_nb_examens_total NUMBER;
106     -- Nombre d'examens effectivement pass s par l' tudiant
107     v_nb_passes_total NUMBER;
108     -- Moyenne calcul e (pond r e par les coefficients)
109     v_moyenne NUMBER;
110 BEGIN
111     -- Total des examens li s aux modules o l' tudiant est
112     inscrit
113     SELECT COUNT(e.id_examen)
114     INTO v_nb_examens_total
115     FROM examen e
116     JOIN inscription i
117     ON e.code_module = i.code_module
118     AND i.id_etudiant = :NEW.id_etudiant;
119
120     -- Nombre d'examens que l' tudiant a r ellement pass
121     SELECT COUNT(*)
122     INTO v_nb_passes_total
123     FROM passer
124     WHERE id_etudiant = :NEW.id_etudiant;
125
126     IF v_nb_passes_total < v_nb_examens_total THEN
127         -- L' tudiant n'a pas encore pass tous ses examens
128         -- moyenne reste NULL, on journalise un
129         avertissement
130
131         UPDATE etudiant
132         SET moyenne = NULL
133         WHERE id_etudiant = :NEW.id_etudiant;
134
135         INSERT INTO journal_erreurs (id_etudiant, message)
136         VALUES (
137             :NEW.id_etudiant,
138             'AVERTISSEMENT C12 : Impossible de calculer la
139             moyenne de l'' tudiant ' ||
140             :NEW.id_etudiant || '. Examens pass s : ' ||
141             v_nb_passes_total ||
142             ' / ' || v_nb_examens_total || '. Moyenne
143             conserv e NULL.'
144         );
145     ELSE

```

```

142      -- L' étudiant a passé tous ses examens      calcul de
           la moyenne
143      -- pondéré par les coefficients de chaque examen
144      SELECT SUM(p.note * e.coefficient) / NULLIF(SUM(e.
           coefficient), 0)
145      INTO    v_moyenne
146      FROM    passer      p
147      JOIN    examen      e ON p.id_examen = e.id_examen
148      JOIN    inscription i
           ON e.code_module = i.code_module
149      AND i.id_etudiant = p.id_etudiant
150      WHERE   p.id_etudiant = :NEW.id_etudiant;
151
152
153      UPDATE etudiant
154      SET     moyenne = ROUND(v_moyenne, 2)
155      WHERE   id_etudiant = :NEW.id_etudiant;
156
157      DBMS_OUTPUT.PUT_LINE(
158          'INFO C12 : Moyenne de l'' étudiant ' || :NEW.
           id_etudiant ||
159          ' calculé avec succès : ' || ROUND(v_moyenne,
           2)
160      );
161
162      END IF;
163
164      END trg_calcul_moyenne_etudiant;
165      /
166
167      -- CONTRAINTE 13 :
168      CREATE OR REPLACE TRIGGER trg_verif_suppression_module
169      BEFORE DELETE ON module
170      FOR EACH ROW
171      DECLARE
172          v_nb_refs          NUMBER;
173          v_modules_dep      VARCHAR2(500) := '';
174
175      BEGIN
176          -- Vérifier si ce module apparaît comme prérequis d'
           autres modules
177          SELECT COUNT(*)
178          INTO    v_nb_refs
179          FROM    prerequis
180          WHERE   prerequis = :OLD.code_module;
181
182          IF v_nb_refs > 0 THEN
183
184              -- Construire la liste des modules qui dépendent de
           celui-ci
185              FOR rec IN (
186                  SELECT module
187                  FROM    prerequis
188                  WHERE   prerequis = :OLD.code_module

```

```

189         ) LOOP
190             v_modules_dep := v_modules_dep || rec.module || '
191             ';
192         END LOOP;
193
194         RAISE_APPLICATION_ERROR(
195             -20013,
196             'ERREUR C13 : Le module ' || :OLD.code_module ||
197             ' ne peut pas tre supprim car il est
198             pr requis de : ' ||
199             TRIM(v_modules_dep) ||
200             '. Supprimez d''abord ces d pendants.'
201         );
202
203     END IF;
204
205 END trg_verif_suppression_module;
206 /
207
208 --
209 =====
210
211 -- TESTS DES TRIGGERS
212 --
213 =====
214
215 -- Donn es de test de base
216 INSERT INTO etudiant VALUES (1, 'DUPONT', 'Alice', SYSDATE,
217     NULL);
218 INSERT INTO etudiant VALUES (2, 'MARTIN', 'Bob', SYSDATE,
219     NULL);
220
221 INSERT INTO module VALUES ('INF101', 'Algorithmique', 30, 10)
222 ;
223 INSERT INTO module VALUES ('INF102', 'Base de donn es', 25,
224     12);
225 INSERT INTO module VALUES ('INF103', 'R seaux', 20,
226     10);
227
228 INSERT INTO prerequis VALUES ('INF102', 'INF101', 10);
229
230 INSERT INTO examen VALUES (1, 'INF101', DATE '2024-03-15', 2);
231 INSERT INTO examen VALUES (2, 'INF101', DATE '2024-04-10', 3);
232 INSERT INTO examen VALUES (3, 'INF102', DATE '2024-05-20', 2);
233
234 COMMIT;
235
236 -- ---- Test C10 : tudiant 1 valide INF101 puis tente de se
237     r inscrire ----
238 INSERT INTO inscription VALUES (1, 'INF101', SYSDATE);
239

```

```

230 -- L' tudiant 1 passe les deux examens d'INF101 avec de
      bonnes notes
231 INSERT INTO passer VALUES (1, 1, 14); -- note 14, coeff 2
232 INSERT INTO passer VALUES (1, 2, 13); -- note 13, coeff 3
233 -- Moyenne = (14*2 + 13*3)/(2+3) = 67/5 = 13.4 => >= 10 =>
      module valid
234
235 -- Cette insertion doit d clencher l'erreur C10 :
236 -- INSERT INTO inscription VALUES (1, 'INF101', SYSDATE);
237
238 -- ---- Test C11 : examen trop proche d'un examen existant
      -----
239 -- L'examen 1 est le 15/03/2024. Celui-ci est le 18/03/2024 (3
      jours apr s)
240 -- => doit d clencher l'erreur C11 :
241 -- INSERT INTO examen VALUES (99, 'INF101', DATE '2024-03-18',
      1);
242
243 -- ---- Test C12 : v rification automatique apr s chaque
      note saisie -----
244 INSERT INTO inscription VALUES (2, 'INF101', SYSDATE);
245 INSERT INTO passer VALUES (2, 1, 8); -- Bob passe 1er
      examen sur 2
246 -- => C12 journalise un avertissement, moyenne reste NULL
247 -- INSERT INTO passer VALUES (2, 2, 11); -- D commenter pour
      compl ter
248
249 -- ---- Test C13 : suppression d'un module pr requis
      -----
250 -- INF101 est pr requis d'INF102 => doit d clencher l'erreur
      C13 :
251 -- DELETE FROM module WHERE code_module = 'INF101';
252
253 -- Affichage du journal des erreurs
254 SELECT * FROM journal_erreurs ORDER BY date_log DESC;
255
256 COMMIT;

```

TP Curseur

```
1  --1.
2  SET SERVEROUTPUT ON;
3
4  DECLARE
5      CURSOR c_cmd IS
6          SELECT *
7          FROM Commande;
8  BEGIN
9      FOR v_cmd IN c_cmd LOOP
10         DBMS_OUTPUT.PUT_LINE(
11             'NumCmd : ' || v_cmd.Numcmd ||
12             ' | Date : ' || v_cmd.Datcmd ||
13             ' | Fournisseur : ' || v_cmd.Numfour ||
14             ' | Montant : ' || v_cmd.Mncmd
15         );
16     END LOOP;
17 END;
18 /
19 --2.
20 SET SERVEROUTPUT ON;
21
22 DECLARE
23     v_moy NUMBER;
24
25     CURSOR c_cmd IS
26         SELECT *
27         FROM Commande
28         WHERE Mncmd >
29             (SELECT AVG(Mncmd)
30              FROM Commande);
31 BEGIN
32     SELECT AVG(Mncmd)
33     INTO v_moy
34     FROM Commande;
35
36     DBMS_OUTPUT.PUT_LINE('Moyenne = ' || v_moy);
37
38     FOR v_cmd IN c_cmd LOOP
39         DBMS_OUTPUT.PUT_LINE(
40             'Commande : ' || v_cmd.Numcmd ||
41             ' | Montant : ' || v_cmd.Mncmd
42         );
43     END LOOP;
44 END;
45 /
46
47 --3.
48 SET SERVEROUTPUT ON;
49
50 DECLARE
51     CURSOR c_paris IS
52         SELECT c.*
```



```

53         FROM Commande c, Fournisseurs f
54         WHERE c.Numfour = f.Numfour
55         AND f.Adrfour = 'Paris';
56 BEGIN
57     FOR v_cmd IN c_paris LOOP
58         DBMS_OUTPUT.PUT_LINE(
59             'Commande : ' || v_cmd.Numcmd ||
60             ' | Montant : ' || v_cmd.Mncmd
61         );
62     END LOOP;
63 END;
64 /
65
66 EXERCICE 2 :
67 --1.
68 DECLARE
69     CURSOR c_emp IS
70         SELECT nom, commission
71         FROM Employe
72         WHERE commission IS NOT NULL
73         ORDER BY commission DESC;
74 BEGIN
75     FOR v_emp IN c_emp LOOP
76         DBMS_OUTPUT.PUT_LINE(
77             v_emp.nom || ' --> ' || v_emp.commission
78         );
79     END LOOP;
80 END;
81 /
82
83 --2. DECLARE
84     CURSOR c_emp IS
85         SELECT nom, date_embauche
86         FROM Employe
87         WHERE date_embauche >= TO_DATE('01/09/2006', 'DD/MM/
88             YYYY');
89 BEGIN
90     FOR v_emp IN c_emp LOOP
91         DBMS_OUTPUT.PUT_LINE(
92             v_emp.nom || ' | ' || v_emp.date_embauche
93         );
94     END LOOP;
95 END;
96 /
97 --3.
98 DECLARE
99     CURSOR c_emp IS
100         SELECT e.nom
101         FROM Employe e, Service s
102         WHERE e.numservice = s.numservice
103         AND s.ville = 'Cr teil';
104 BEGIN
105     FOR v_emp IN c_emp LOOP

```

```

105         DBMS_OUTPUT.PUT_LINE(v_emp.nom);
106     END LOOP;
107 END;
108 /
109
110 --4.
111 DECLARE
112     CURSOR c_sub IS
113         SELECT e.nom
114         FROM Employe e
115         WHERE e.superieur =
116             (SELECT numemp
117              FROM Employe
118              WHERE nom = 'Guimezanes');
119 BEGIN
120     FOR v_emp IN c_sub LOOP
121         DBMS_OUTPUT.PUT_LINE(v_emp.nom);
122     END LOOP;
123 END;
124 /
125
126 --5.
127 DECLARE
128     v_moy NUMBER;
129 BEGIN
130     SELECT AVG(salaire)
131     INTO v_moy
132     FROM Employe;
133
134     DBMS_OUTPUT.PUT_LINE(
135         'Moyenne des salaires = ' || v_moy
136     );
137 END;
138 /
139
140 --6.
141 DECLARE
142     v_nb NUMBER;
143 BEGIN
144     SELECT COUNT(*)
145     INTO v_nb
146     FROM Employe
147     WHERE commission IS NOT NULL;
148
149     DBMS_OUTPUT.PUT_LINE(
150         'Nombre de commissions = ' || v_nb
151     );
152 END;
153 /
154 --7.
155 DECLARE
156     CURSOR c_emp IS
157         SELECT nom, salaire

```

```

158         FROM Employe
159         WHERE salaire >
160             (SELECT AVG(salaire)
161              FROM Employe);
162
163     v_moy NUMBER;
164 BEGIN
165     SELECT AVG(salaire)
166     INTO v_moy
167     FROM Employe;
168
169     DBMS_OUTPUT.PUT_LINE(
170         'Moyenne = ' || v_moy
171     );
172
173     FOR v_emp IN c_emp LOOP
174         DBMS_OUTPUT.PUT_LINE(
175             v_emp.nom || ' --> ' || v_emp.salaire
176         );
177     END LOOP;
178 END;
179 /

```

TP PL/SQL : IF-THEN-ELSE et CURSEURS

```
1
2 SET SERVEROUTPUT ON;
3
4
5 -- Exercice 1
6 DECLARE
7     small_nbr NUMBER := 25;
8     big_nbr   NUMBER := 5;
9     tmp       NUMBER;
10 BEGIN
11     DBMS_OUTPUT.PUT_LINE('=== Exercice IF 1 :   change   de
12                          variables ===');
13     DBMS_OUTPUT.PUT_LINE('Avant : small_nbr = ' || small_nbr
14                          || ', big_nbr = ' || big_nbr);
15
16     IF small_nbr > big_nbr THEN
17         tmp := small_nbr;
18         small_nbr := big_nbr;
19         big_nbr := tmp;
20     END IF;
21
22     DBMS_OUTPUT.PUT_LINE('Apr s : small_nbr = ' || small_nbr
23                          || ', big_nbr = ' || big_nbr);
24 END;
25 /
26
27 -- Exercice 2
28 DECLARE
29     nbr NUMBER := 3;
30 BEGIN
31     DBMS_OUTPUT.PUT_LINE('=== Exercice IF 2 : Pair ou Impair
32                          ===');
33
34     IF MOD(nbr, 2) = 0 THEN
35         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est un
36                               nombre pair.');
```

```

majuscules (selon NLS)
46 jour_sem := TRIM(TO_CHAR(date1, 'DAY', 'NLS_DATE_LANGUAGE=
    ENGLISH')));
47
48 IF jour_sem IN ('SATURDAY', 'SUNDAY') THEN
49     DBMS_OUTPUT.PUT_LINE(
50         'Le jour de la date donn e est ' || jour_sem || '
            : tombe le week-end.'
51     );
52 ELSE
53     DBMS_OUTPUT.PUT_LINE(
54         'Le jour de la date donn e est ' || jour_sem || '
            : jour ouvr .'
55     );
56 END IF;
57 END;
58 /
59
60 -- Exercice 4 : Copier les enregistrements de clients vers
    tmp_clients
61
62 -- Cr ation des tables
63 DROP TABLE tmp_clients CASCADE CONSTRAINTS;
64 DROP TABLE clients      CASCADE CONSTRAINTS;
65
66 CREATE TABLE clients (
67     client_id NUMBER PRIMARY KEY,
68     nom        VARCHAR2(100),
69     ville      VARCHAR2(50),
70     age        NUMBER
71 );
72
73 CREATE TABLE tmp_clients (
74     client_id NUMBER PRIMARY KEY,
75     nom        VARCHAR2(100),
76     ville      VARCHAR2(50),
77     age        NUMBER
78 );
79
80 -- Donn es initiales
81 INSERT INTO clients VALUES (1001, 'Alex Barbara', 'Paris',
    25);
82 INSERT INTO clients VALUES (1002, 'Lois Collins', 'Lyon',
    30);
83 INSERT INTO clients VALUES (1005, 'Karen Farley', 'Nice',
    44);
84 INSERT INTO clients VALUES (1006, 'Kevin Thompson', 'Paris',
    65);
85 INSERT INTO clients VALUES (1007, 'Scott Henry', 'Marseille
    ', 32);
86 INSERT INTO clients VALUES (1003, 'Ellen McMullen', '
    Montpellier', 53);
87 COMMIT;

```

```

88
89 -- Bloc PL/SQL de copie
90 DECLARE
91     v_nb_copies NUMBER := 0;
92 BEGIN
93     DBMS_OUTPUT.PUT_LINE('=== Exercice IF 4 : Copie clients
94                             tmp_clients ===');
95
96     FOR rec IN (SELECT * FROM clients) LOOP
97         INSERT INTO tmp_clients (client_id, nom, ville, age)
98         VALUES (rec.client_id, rec.nom, rec.ville, rec.age);
99         v_nb_copies := v_nb_copies + 1;
100     END LOOP;
101
102     COMMIT;
103     DBMS_OUTPUT.PUT_LINE(v_nb_copies || ' enregistrement(s)
104                             copi (s) dans tmp_clients. ');
105
106 END;
107 /
108
109 -- Cr ation des tables n cessaires
110 DROP TABLE pays          CASCADE CONSTRAINTS;
111 DROP TABLE employees     CASCADE CONSTRAINTS;
112 DROP TABLE departements  CASCADE CONSTRAINTS;
113
114 CREATE TABLE pays (
115     pays_id NUMBER PRIMARY KEY,
116     nom      VARCHAR2(50)
117 );
118
119 CREATE TABLE departements (
120     dep_id NUMBER PRIMARY KEY,
121     nom     VARCHAR2(50)
122 );
123
124 CREATE TABLE employees (
125     id          NUMBER PRIMARY KEY,
126     nom         VARCHAR2(50),
127     prenom      VARCHAR2(50),
128     salaire     NUMBER(10,2),
129     departement_id NUMBER REFERENCES departements(dep_id)
130 );
131
132 -- Donn es
133 INSERT INTO pays VALUES (1001, 'Angola');
134 INSERT INTO pays VALUES (1002, 'Australia');
135 INSERT INTO pays VALUES (1005, 'Albania');
136 INSERT INTO pays VALUES (1006, 'Bangladesh');
137 INSERT INTO pays VALUES (1007, 'Andorra');
138 INSERT INTO pays VALUES (1003, 'Bolivia');

```

```

139 INSERT INTO departements VALUES (1, 'Marketing');
140 INSERT INTO departements VALUES (2, 'Finance');
141 INSERT INTO departements VALUES (3, 'HR');
142 INSERT INTO departements VALUES (4, 'Informatique');
143 INSERT INTO departements VALUES (5, 'Exp ditions');
144 INSERT INTO departements VALUES (6, 'Administration');
145 INSERT INTO departements VALUES (8, 'D partement 8');
146 INSERT INTO departements VALUES (9, 'D partement 9');
147
148 INSERT INTO employees VALUES (1001, 'Eddie', 'Parker',
    6000.00, 8);
149 INSERT INTO employees VALUES (1002, 'Eleanor', 'Deas',
    4500.00, 8);
150 INSERT INTO employees VALUES (1003, 'Glen', 'Powell',
    5000.00, 6);
151 INSERT INTO employees VALUES (1004, 'Ali', 'Fawaz',
    7800.00, 2);
152 INSERT INTO employees VALUES (1005, 'Earl', 'Horn',
    9000.00, 9);
153 INSERT INTO employees VALUES (1006, 'Bryan', 'Savoy',
    8000.00, 3);
154 COMMIT;
155
156
157 -- Curseur 1
158 DECLARE
159     CURSOR cur_pays IS
160         SELECT nom FROM pays;
161
162     v_nom pays.nom%TYPE;
163 BEGIN
164     DBMS_OUTPUT.PUT_LINE('=== Curseur 1 : Noms de tous les
    pays ===');
165
166     OPEN cur_pays;
167     LOOP
168         FETCH cur_pays INTO v_nom;
169         EXIT WHEN cur_pays%NOTFOUND;
170         DBMS_OUTPUT.PUT_LINE(' - ' || v_nom);
171     END LOOP;
172     CLOSE cur_pays;
173 END;
174 /
175
176
177 -- Curseur 2
178 DECLARE
179     CURSOR cur_pays IS
180         SELECT pays_id, nom FROM pays;
181
182     v_id pays.pays_id%TYPE;
183     v_nom pays.nom%TYPE;
184 BEGIN

```

```

185 DBMS_OUTPUT.PUT_LINE('=== Curseur 2 : ID et noms des pays
    ===');
186 DBMS_OUTPUT.PUT_LINE(RPAD('Identifiant', 14) || 'Nom du
    pays');
187 DBMS_OUTPUT.PUT_LINE(RPAD('-----', 14) || '
    -----');
188
189 OPEN cur_pays;
190 LOOP
191     FETCH cur_pays INTO v_id, v_nom;
192     EXIT WHEN cur_pays%NOTFOUND;
193     DBMS_OUTPUT.PUT_LINE(RPAD(v_id, 14) || v_nom);
194 END LOOP;
195 CLOSE cur_pays;
196 END;
197 /
198
199
200 -- Curseur 3 :
201 BEGIN
202     DBMS_OUTPUT.PUT_LINE('=== Curseur 3 : SQL%ROWCOUNT ===');
203
204     -- Exemple : mise      jour des pays dont le nom commence
        par 'A'
205     UPDATE pays
206     SET     nom = UPPER(nom)
207     WHERE  nom LIKE 'A%';
208
209     DBMS_OUTPUT.PUT_LINE(
210         SQL%ROWCOUNT || ' ligne(s) affect e(s) par l''UPDATE.
        ,
211     );
212
213     ROLLBACK;
214 END;
215 /
216
217
218 -- Curseur 5 : Afficher id, nom, pr nom et d departement de
        chaque employ
219 DECLARE
220     CURSOR cur_emp IS
221         SELECT e.id, e.nom, e.prenom, d.nom AS nom_dept
222         FROM   employees     e
223         JOIN   departements d ON e.departement_id = d.dep_id
224         ORDER BY e.id;
225
226     v_id      employees.id%TYPE;
227     v_nom      employees.nom%TYPE;
228     v_prenom   employees.prenom%TYPE;
229     v_dept     departements.nom%TYPE;
230 BEGIN
231     DBMS_OUTPUT.PUT_LINE('=== Curseur 5 : Employ s et leurs

```



```

232         d departements ===');
233 DBMS_OUTPUT.PUT_LINE(
234     RPAD('ID', 6) || RPAD('Nom', 12) ||
235     RPAD('Pr nom', 12) || 'D departement'
236 );
237 DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
238
239 OPEN cur_emp;
240 LOOP
241     FETCH cur_emp INTO v_id, v_nom, v_prenom, v_dept;
242     EXIT WHEN cur_emp%NOTFOUND;
243     DBMS_OUTPUT.PUT_LINE(
244         RPAD(v_id, 6) || RPAD(v_nom, 12) ||
245         RPAD(v_prenom, 12) || v_dept
246     );
247 END LOOP;
248 CLOSE cur_emp;
249 END;
250 /
251
252 -- Curseur 6 :
253 BEGIN
254     DBMS_OUTPUT.PUT_LINE('=== Curseur 6 : SQL%FOUND apr s
255         DELETE ===');
256
257     DELETE FROM pays WHERE pays_id = 1001; -- Angola
258
259     IF SQL%FOUND THEN
260         DBMS_OUTPUT.PUT_LINE(
261             'DELETE r ussi : ' || SQL%ROWCOUNT || ' ligne(s)
262             supprim e(s).'
263         );
264     ELSE
265         DBMS_OUTPUT.PUT_LINE('DELETE : aucune ligne supprim e
266             .');
267     END IF;
268
269     ROLLBACK;
270
271     -- Test avec un identifiant inexistant
272     DELETE FROM pays WHERE pays_id = 9999;
273
274     IF SQL%FOUND THEN
275         DBMS_OUTPUT.PUT_LINE('DELETE r ussi.');
```

```

280
281 -- Curseur 7 :
282 BEGIN
283     DBMS_OUTPUT.PUT_LINE('=== Curseur 7 : SQL%NOTFOUND apr s
        UPDATE ===');
284
285     -- Test 1 : mise      jour d'un pays existant
286     UPDATE pays SET nom = 'Angola (mis      jour)' WHERE pays_id
        = 1001;
287
288     IF SQL%NOTFOUND THEN
289         DBMS_OUTPUT.PUT_LINE('UPDATE : aucune ligne mise
            jour pour pays_id = 1001.');
```

```

290     ELSE
291         DBMS_OUTPUT.PUT_LINE(
292             'UPDATE r ussi : ' || SQL%ROWCOUNT || ' ligne(s)
                mise(s)      jour.'
293         );
294     END IF;
295
296     ROLLBACK;
297
298     -- Test 2 : mise      jour d'un pays inexistant
299     UPDATE pays SET nom = 'Inconnu' WHERE pays_id = 9999;
300
301     IF SQL%NOTFOUND THEN
302         DBMS_OUTPUT.PUT_LINE('UPDATE : aucune ligne trouv e
            pour pays_id = 9999.');
```

```

303     ELSE
304         DBMS_OUTPUT.PUT_LINE('UPDATE r ussi.');
```

```

305     END IF;
306
307     ROLLBACK;
308 END;
309 /
310
311
312 -- Curseur 8 :
313 DECLARE
314     -- Curseur FOR UPDATE pour permettre WHERE CURRENT OF
315     CURSOR cur_emp_dept8 IS
316         SELECT id, nom, prenom, salaire
317         FROM   employees
318         WHERE  departement_id = 8
319         FOR UPDATE OF salaire;
320
321     v_nouveau_salaire employees.salaire%TYPE;
322 BEGIN
323     DBMS_OUTPUT.PUT_LINE('=== Curseur 8 : Augmentation
        salaires d partement 8 ===');
```

```

324     DBMS_OUTPUT.PUT_LINE(RPAD('Employ ', 20) || RPAD('Ancien
        salaire', 18) || 'Nouveau salaire');
```

```

325     DBMS_OUTPUT.PUT_LINE(RPAD('-', 55, '-'));
```

```

326
327     FOR rec IN cur_emp_dept8 LOOP
328
329         IF rec.salaire < 5000 THEN
330             v_nouveau_salaire := rec.salaire + 100;
331         ELSE
332             v_nouveau_salaire := rec.salaire + 50;
333         END IF;
334
335         -- Mise      jour directe sur la ligne courante du
336             curseur
337         UPDATE employees
338         SET      salaire = v_nouveau_salaire
339         WHERE CURRENT OF cur_emp_dept8;
340
341         DBMS_OUTPUT.PUT_LINE(
342             RPAD(rec.nom || ' ' || rec.prenom, 20) ||
343             RPAD(rec.salaire || ' $', 18) ||
344             v_nouveau_salaire || ' $'
345         );
346
347     END LOOP;
348
349     COMMIT;
350     DBMS_OUTPUT.PUT_LINE('Mise      jour termin e et valid e (
351         COMMIT).');
352
353 END;
354 /

```

```

1 SET SERVEROUTPUT ON;
2
3 -- EXERCICE 1
4
5 BEGIN
6     DBMS_OUTPUT.PUT_LINE('Hello');
7 END;
8 /
9
10 CREATE OR REPLACE FUNCTION carre(n NUMBER)
11 RETURN NUMBER
12 IS
13 BEGIN
14     RETURN n*n;
15 END;
16 /
17
18 SELECT carre(5) FROM DUAL;
19
20 -- EXERCICE 2
21
22 CREATE OR REPLACE PROCEDURE maximum2(
23     a IN NUMBER,
24     b IN NUMBER,
25     res OUT NUMBER
26 )
27 IS
28 BEGIN
29     IF a>b THEN
30         res:=a;
31     ELSE
32         res:=b;
33     END IF;
34 END;
35 /
36
37 DECLARE
38     m NUMBER;
39 BEGIN
40     maximum2(10,20,m);
41     DBMS_OUTPUT.PUT_LINE('Maximum = '||m);
42 END;
43 /
44
45 CREATE OR REPLACE FUNCTION maximum2f(
46     a NUMBER,
47     b NUMBER
48 )
49 RETURN NUMBER
50 IS
51 BEGIN
52     IF a>b THEN

```

```

53         RETURN a;
54     ELSE
55         RETURN b;
56     END IF;
57 END;
58 /
59
60 -- EXERCICE 3
61
62 CREATE OR REPLACE PROCEDURE permuter(
63     a IN OUT NUMBER,
64     b IN OUT NUMBER
65 )
66 IS
67     temp NUMBER;
68 BEGIN
69     temp:=a;
70     a:=b;
71     b:=temp;
72 END;
73 /
74
75 DECLARE
76     x NUMBER:=10;
77     y NUMBER:=20;
78 BEGIN
79     permuter(x,y);
80     DBMS_OUTPUT.PUT_LINE('x='||x||' y='||y);
81 END;
82 /
83
84 -- EXERCICE 4
85
86 CREATE OR REPLACE PROCEDURE augmenter_salaire(
87     p_nom VARCHAR2,
88     p_taux NUMBER
89 )
90 IS
91 BEGIN
92     UPDATE Employe
93     SET salaire=salaire*(1+p_taux/100)
94     WHERE nom=p_nom;
95
96     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT||' ligne(s) modifi e(s)
97         ');
98 END;
99 /
100 CREATE OR REPLACE PROCEDURE augmenter_salaire_id(
101     p_enum NUMBER,
102     p_taux NUMBER
103 )
104 IS

```

```

105 BEGIN
106     UPDATE Employe
107     SET salaire=salaire*(1+p_taux/100)
108     WHERE enum=p_enum;
109
110     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT||' ligne(s) modifi e(s)
111     ');
112 END;
113 /
114
115 -- EXERCICE 5
116
117
118 CREATE OR REPLACE FUNCTION mad_to_euro(
119     montant NUMBER
120 )
121 RETURN NUMBER
122 IS
123 BEGIN
124     RETURN ROUND(montant/11,2);
125 END;
126 /
127
128 CREATE OR REPLACE FUNCTION mad_to_euro(
129     montant NUMBER,
130     taux NUMBER
131 )
132 RETURN NUMBER
133 IS
134 BEGIN
135     RETURN ROUND(montant/taux,2);
136 END;
137 /
138
139 SELECT mad_to_euro(1100) FROM DUAL;
140 SELECT mad_to_euro(1100,10) FROM DUAL;
141
142
143 -- EXERCICE 6
144
145
146 CREATE OR REPLACE PROCEDURE afficher_salaires(
147     p_montant NUMBER
148 )
149 IS
150 BEGIN
151     FOR emp IN (
152         SELECT nom,salaire
153         FROM Employe
154         WHERE salaire>p_montant
155     )
156     LOOP

```

```

157         DBMS_OUTPUT.PUT_LINE(
158             emp.nom||' : '||emp.salaire
159         );
160     END LOOP;
161 END;
162 /
163
164
165 -- EXERCICE 7
166
167 CREATE OR REPLACE FUNCTION salaire_moyen(
168     p_dep NUMBER
169 )
170 RETURN NUMBER
171 IS
172     v_moy NUMBER;
173 BEGIN
174     SELECT AVG(salaire)
175     INTO v_moy
176     FROM Employe
177     WHERE numdep=p_dep;
178
179     RETURN v_moy;
180 END;
181 /
182
183
184 -- EXERCICE 8
185
186 CREATE OR REPLACE PROCEDURE modifier_adresse(
187     p_nom VARCHAR2,
188     p_adresse VARCHAR2
189 )
190 IS
191 BEGIN
192     IF p_adresse IS NULL THEN
193         RAISE_APPLICATION_ERROR(
194             -20001,
195             'Adresse vide'
196         );
197     END IF;
198
199     UPDATE Employe
200     SET adresse=p_adresse
201     WHERE nom=p_nom;
202
203     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT||' ligne(s) modifi e(s)
204         ');
205 END;
206 /
207
208 CREATE OR REPLACE PROCEDURE modifier_adresse_id(
209     p_enum NUMBER,

```

```

209     p_adresse VARCHAR2
210 )
211 IS
212 BEGIN
213     UPDATE Employe
214     SET adresse=p_adresse
215     WHERE enum=p_enum;
216
217     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT||' ligne(s) modifi e(s)
218     ');
219 END;
220 /
221
222 -- EXERCICE 9
223
224 CREATE OR REPLACE FUNCTION nb_employes(
225     p_dep NUMBER
226 )
227 RETURN NUMBER
228 IS
229     v_nb NUMBER;
230 BEGIN
231     SELECT COUNT(*)
232     INTO v_nb
233     FROM Employe
234     WHERE numdep=p_dep;
235
236     RETURN v_nb;
237 END;
238 /
239
240 CREATE OR REPLACE PROCEDURE afficher_effectifs
241 IS
242 BEGIN
243     FOR d IN (
244         SELECT numdep,
245                COUNT(*) nb
246         FROM Employe
247         GROUP BY numdep
248     )
249     LOOP
250         DBMS_OUTPUT.PUT_LINE(
251             'D partement '||
252             d.numdep||
253             ' : '||
254             d.nb
255         );
256     END LOOP;
257 END;
258 /
259
260

```



```

261  -- EXERCICE 10
262
263
264  CREATE OR REPLACE PROCEDURE supprimer_petits_salaires(
265      p_montant NUMBER
266  )
267  IS
268  BEGIN
269      DELETE FROM Employe
270      WHERE salaire < p_montant;
271
272      DBMS_OUTPUT.PUT_LINE(
273          SQL%ROWCOUNT ||
274          ' employ (s) supprim (s)'
275      );
276  END;
277  /
278
279  CREATE OR REPLACE PROCEDURE supprimer_petits_salaires_sp(
280      p_montant NUMBER
281  )
282  IS
283  BEGIN
284      SAVEPOINT avant_suppr;
285
286      DELETE FROM Employe
287      WHERE salaire < p_montant;
288
289  EXCEPTION
290      WHEN OTHERS THEN
291          ROLLBACK TO avant_suppr;
292          DBMS_OUTPUT.PUT_LINE(SQLERRM);
293  END;
294  /

```